Class Photos

AlgoExpert · Easy · Greedy Algorithms

The problem

- Equal numbers of red and blue shirted students; all of one colour in the back row, each back-row student strictly taller than the person in front.
- Is a photo possible?

Intuition

- Sort both groups.
- The taller group must stand in the back, and then every back student must be taller than the one in front.

Brute force vs optimal

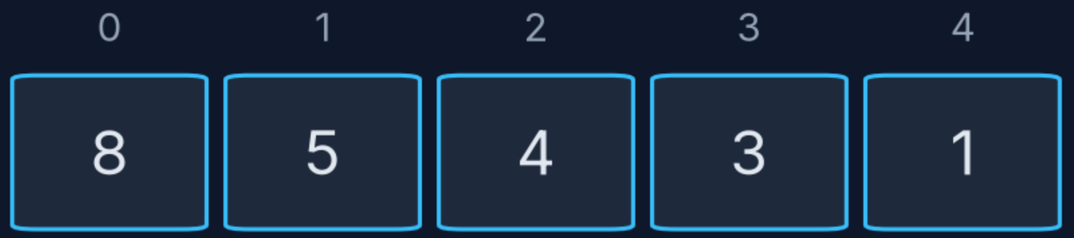
- Brute force: try every arrangement → factorial time
- Optimal: sort both, compare pairwise → O(n log n)
- Pairing sorted with sorted is the best possible matching

Approach

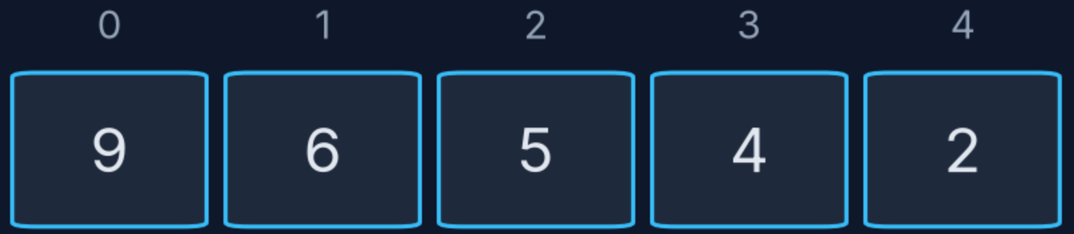
- 1. Sort red and blue shirts
- 2. The group with the taller tallest goes in the back
- 3. Check every pair strictly
- 4. Any failure means false

Worked example

red (sorted desc)



blue (sorted desc)



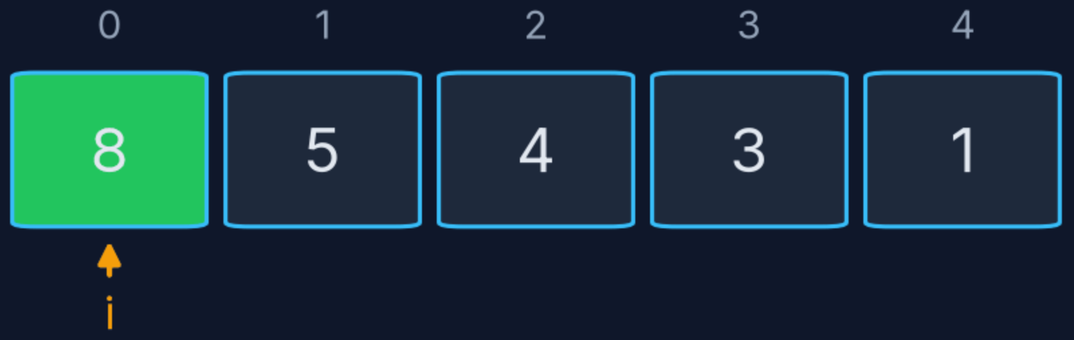
Sort both groups; tallest blue 9 > tallest red 8

Worked example

back row (blue)



front row (red)



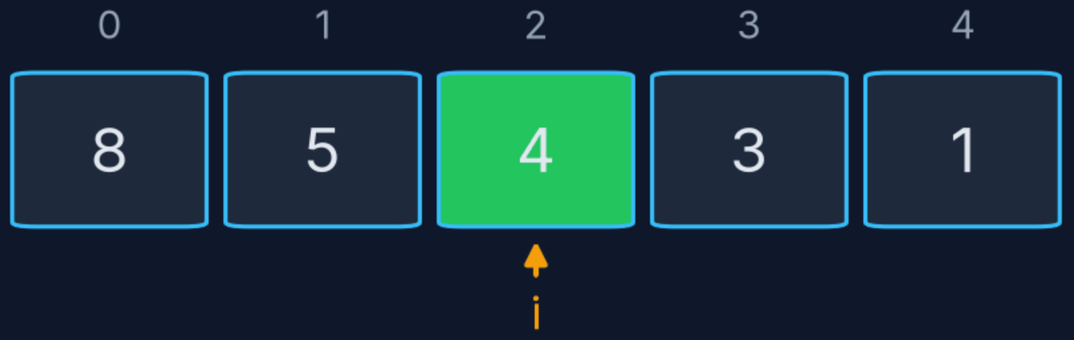
9 > 8 ✓

Worked example

back row (blue)



front row (red)



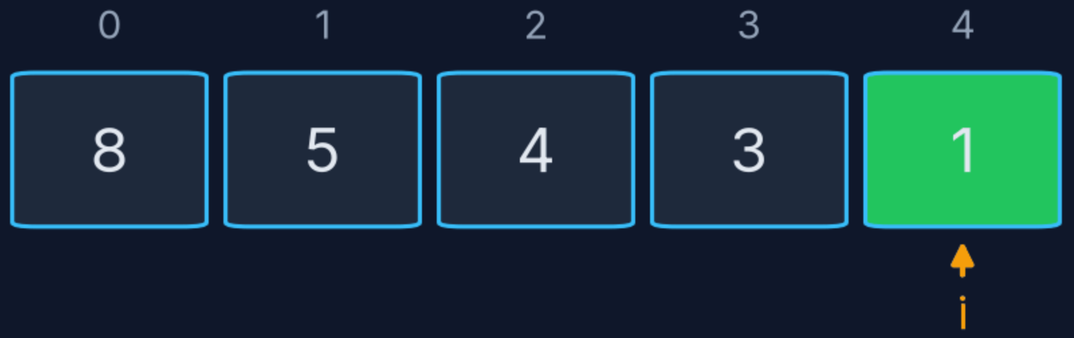
5 > 4 ✓

Worked example

back row (blue)



front row (red)



2 > 1 ✓

Java solution

```java
public class ClassPhotos {
    public static boolean classPhotos(List<Integer> red, List<Integer> blue) {
        Collections.sort(red, Collections.reverseOrder());
        Collections.sort(blue, Collections.reverseOrder());
        boolean redBack = red.get(0) > blue.get(0);
        for (int i = 0; i < red.size(); i++) {
            int back = redBack ? red.get(i) : blue.get(i), front = redBack ? blue.get(i) : red.get(i);
            if (back <= front) return false;
        }
        return true;
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(1)

Edge cases & pitfalls

- Strictly taller required: equal heights fail
- Decide back row from the tallest of each group
- Groups have equal size

Interview tips & follow-ups

- Explain why sorted pairing is optimal (exchange argument)
- Follow-up: count arrangements, or allow mixed rows

Recap

- Sort both groups.
- Complexity: O(n log n)